Table of Contents

1. Introduction 1

2. Problem Statement 2

3. Objectives 3

4. Methodology 4

4.1. Requirement Identification 4

4.1.1. Literature Review 4

4.1.2. Study of Existing System 5

4.2. Requirement Analysis 6

4.3. Feasibility Study 7

4.3.1. Technical 7

4.3.2. Operational 8

4.3.3. Economic 8

4.3.4. Schedule 8

4.4. System Design 9

4.4.1. Applicable Tools and Technologies 9

4.4.2. Applicable Algorithms 11

4.4.2.1. Cosine Similarity Algorithm 11

4.4.2.2. Haversine Formula for Location-Based Filtering 13

4.4.3. Proposed Work 14

4.4.3.1. System Architecture 14

4.4.3.2. Flowchart Diagram 15

4.4.3.3. Use Case Diagram 16

4.4.3.4. MongoDB Data Model 17

5. Expected Outcomes 18

6. References 19

1. Introduction

The rapid growth of internet access and digital services has significantly changed how people search for jobs and how organizations hire employees. Job recruitment processes in Nepal are still largely manual, time-consuming, and dependent on personal networks, social media posts, or basic job listing and application collection websites. These methods often create difficulties for both job seekers and employers. Job seekers struggle to find relevant opportunities that match their skills, experience, while employers face challenges in filtering large numbers of applications and identifying suitable candidates efficiently.

With the increasing number of skilled workers entering the job market every year, there is a strong need for a smarter and more organized recruitment system. A job portal that not only lists jobs but also provide intelligent job recommendations, automated shortlisting, and proper communication between job seeker and employers can greatly improve the hiring process. Such a system can save time, reduce human effort, and provide better transparency in recruitment process.

KARYASETU – Job Portal Platform is developed as a 7th Semester Project for TU CSIT with the aim of addressing these issues in the context of Nepal. The system is designed to act as a bridge between job seekers and employers by using intelligent algorithms and automation. It provides features such as automated resume shortlisting for employers, application status tracking, secure user authentication, premium subscription services, and profile-based job recommendations.

The system focuses on using structured profile data provided by job seekers, such as skills, education, experience, and other details, instead of extracting text from resume documents. This approach ensures accuracy and consistency in matching. By integrating cosine similarity for skill-based matching and the Haversine algorithm for location-based filtering, KARYASETU aims to make job discovery and hiring process more efficient, reliable, and user-friendly.

2. Problem Statement

Despite the availability of several online job portal platforms, the recruitment process in Nepal still faces multiple challenges. Most existing job portals function mainly as notice boards where jobs are posted and applications are collected, without providing intelligent support for matching candidates with suitable jobs. This leads to several problems.

Job seekers often need to manually search through a large number of job listings to find opportunities that match their skills, experience, and preferred location. This process is time consuming and inefficient, especially when application status updates are unclear or unavailable. The lack of timely application status update notifications further increases uncertainty, making it difficult for candidates to track the progress of their applications.

Employers, on the other hand, frequently receive a high volume of applications for each job posting. Reviewing these applications manually requires significant time and effort, increasing the chances of overlooking suitable candidates. Smaller organizations, in particular, may lack the resources to implement advanced recruitment tools like Application Tracking System (ATS) that could improve screening efficiency.

In addition to efficiency issues, existing platforms often lack strong security and moderation mechanisms. Fake accounts, fraudulent job postings, and limited verification processes reduce user trust. Communication between employers and job seekers is also poorly managed in many systems, leading to delays and misunderstandings during the hiring process.

Furthermore, most current systems do not effectively integrate profile-based job recommendations or structured profile data, which are important factors in improving job relevance and practical usability.

Therefore, there is a clear need for a secure and intelligent job portal platform that can automate resume shortlisting, provide personalized skill, education, work experience and location aware job recommendations, offer transparent application tracking, and ensure proper moderation and communication. Addressing these challenges forms the core problem that this project aims to solve.

3. Objectives

The main objective of the KARYASETU project is to design and develop a job portal platform that improves the efficiency and reliability of the recruitment process in Nepal.

The specific objectives of the project are:

- To develop a web-based job portal that connects job seekers and employers through a structured and user-friendly interface.
- To implement an intelligent job application shortlisting system for employers using cosine similarity to rank applicants based on job relevance.
- To provide personalized job recommendations to job seekers based on their skills, experience, education and location.
- To enable job seekers to track the status of their job applications in real time.
- To implement secure user authentication with email OTP verification, password change, and OTP-based password recovery.
- To introduce a premium subscription model that provides additional services such as email notifications and scheduled job recommendations.
- To ensure platform safety through job scam reporting, review comment posting and admin moderation features.
- To provide an admin panel for content management, viewing analytics, and revenue tracking.
- To integrate a secure online payment gateway for subscription management.

4. Methodology

The methodology section explains how the KARYASETU system is planned, analyzed, and designed. It focuses on understanding the problem, identifying system requirements, evaluating feasibility, and selecting appropriate algorithms, techniques and technologies. The overall approach follows a structured software development process to ensure clarity, reliability, and correctness of the system.

4.1. Requirement Identification

Requirement identification is the first and most critical phase of the project development. It involves understanding existing systems, studying relevant research works, and identifying functional and non-functional requirements based on real-world recruitment challenges in Nepal.

4.1.1. Literature Review

In the field of Information Retrieval (IR), representing documents as high-dimensional vectors known as the **Vector Space Model** has proven superior to binary keyword matching. By calculating the **cosine similarity** between a candidate's resume vector and a job description vector, systems can facilitate partial matches and rank results by semantic relevance [1]. Research in e-recruitment confirms that these similarity-based ranking mechanisms significantly enhance the precision and quality of results when processing massive datasets of job descriptions [2].

The integration of automated screening tools has been shown to drastically reduce the administrative burden on human recruiters. Studies on bilateral matching approaches demonstrate that recommendation systems can maintain high selection accuracy while minimizing manual effort [3]. Modern recruitment platforms utilize **personalized recommendation techniques** to suggest roles based on user profiles and explicit preferences, which mitigates "information overload" and reduces the time users spend on manual searches [4].

As location remains a primary constraint in employment, **location-aware services** have become integral to matching algorithms. The **Haversine formula** is the established method for calculating the "great-circle" distance between a user and a job site on spherical Earth [5]. This allows platforms to filter or weight opportunities based on geographic feasibility, which is critical in regions where commuting or relocation is a barrier.

4.1.2. Study of Existing System

Several online job portals and recruitment platforms are currently in use, both globally and within Nepal. These platforms aim to simplify the hiring process by providing digital job listings, application management, and basic communication between employers and job seekers. However, their features and level of automation vary.

LinkedIn is a widely used global platform that integrates professional networking with job listings. It allows employers to post vacancies and view candidate profiles, while job seekers can apply using their professional profiles. The platform provides job suggestions based on profile information and activity, resume screening and candidate evaluation are done by automated process of their own propriety AI model developed internally by their workforce that costed millions of dollars for research and development.

Glassdoor is also a popular global platform that combines job listings with company reviews and salary information. It helps job seekers understand workplace conditions but does not focus heavily on intelligent resume matching or automated candidate ranking. The hiring process still relies mostly on manual application review methods.

In the Nepal, platforms such as **MeroJob** and **JobsNepal** are commonly used. These systems mainly operate as job listing and application collection portal. Job seekers manually search and apply for vacancies, and employers review applications individually. While these platforms have improved access to job information, they offer no support for intelligent matching, automated resume shortlisting, or personalized job recommendations.

4.2. Requirement Analysis

Based on the literature review and problem understanding, the system requirements are analyzed and categorized.

4.2.1. Functional Requirements

- The system must allow users to **register** to system with some details like name, email, password, role and avatar (profile picture), authenticity of user's email account ensured with email verification through One Time Password (OTP).
- If users already have an account, then they must be able to **log in** securely and manage their account information.
- Job seekers must be able to **create** and **update** profiles containing skills, education, experience, location, phone number, resume and other details.
- Employers must be able to **post**, and **manage** job vacancies.
- The system must **calculate similarity scores** between job requirements and job seeker profiles using **cosine similarity**.
- Job seekers must receive **job recommendations** based on profile similarity and geographic distance.
- The system must allow job seekers to **apply** for jobs and **track application status.**
- **Premium users** must **receive email notifications** on application status updates and **scheduled job recommendations**.
- **Employers** and **job seekers** must be able to **communicate** through a chat system during the **interview stage** to plan for interview scheduling and ask further questions as required.
- **Admin** users must be able to **monitor** system activity, **view** jobs posted in this platform and even **delete jobs**, **ban employers if required, review reported jobs**, and **track revenue.**

4.2.2. Non-Functional Requirements

- The system must comply with **data protection regulations** and **digital privacy standards** to ensure data security, user privacy and secure digital transactions.
- The user interface must be **easy** to use and **accessible**.
- The system must be **scalable** to support increasing users and job postings.
- The user interface must be **fully responsive** and **device compatible**, functioning seamlessly across different size devices like mobile, tablet and desktop/laptop.
- The system architecture must be **modular** and **maintainable**.
- The platform should be **available** for **24x7,** ensuring that user can access services whenever required.

4.3. Feasibility Study

The feasibility study evaluates whether the project can be successfully developed and implemented within available resources and constraints.

4.3.1. Technical

The project is technically feasible because it uses proven technologies and algorithms. Cosine similarity is computationally efficient and suitable for matching jobs with structured profile data. The Haversine algorithm is simple to implement and widely used in real-world applications to calculate distance between two locations.

Backend services such as authentication, email delivery, and scheduled tasks can be handled using Node.js, NodeMailer, and Node Cron. Payment integration through eSewa is feasible using available APIs. The system does not require advanced hardware or proprietary software, making development practical within an academic environment.

4.3.2. Operational

From an operational point of view, the system is designed to be straightforward for all users. Job seekers can easily register, build profiles, and apply for jobs. Employers can post vacancies and view ranked applicant lists, update application status without technical expertise. Admin users can manage platform content through a centralized dashboard.

The inclusion of clear workflows, such as application status stages and restricted chat access, ensures smooth system operation and reduces misuse.

4.3.3. Economic

The project is economically feasible because it can be developed using open-source tools and frameworks. Development costs are minimal, limited mainly to time and effort. However, if we further plan to release this system to real word then only it will require us to use paid hosting, domain and email services. But for now, we are only developing this project for academic purpose, so there is no cost of development. The premium subscription model demonstrates how the system could generate revenue if deployed in a real-world setting, while still offering essential services to free users.

4.3.4. Schedule

The project will be developed over a period of 120 days with tasks being divided along with the duration in the time frame. In order for the system to be delivered in the specified time frame, we made sure that deadlines can be met and tasks can be executed accordingly in the specified duration period of our 7th semester. For the development of this project, we will use Waterfall Development Model, it is a **linear software development process** where each phase is completed one after another (Requirements Gathering → Planning→ Designing → Coding → Testing and debugging→ Documentation→ Maintenance). This schedule is designed as per this development model.

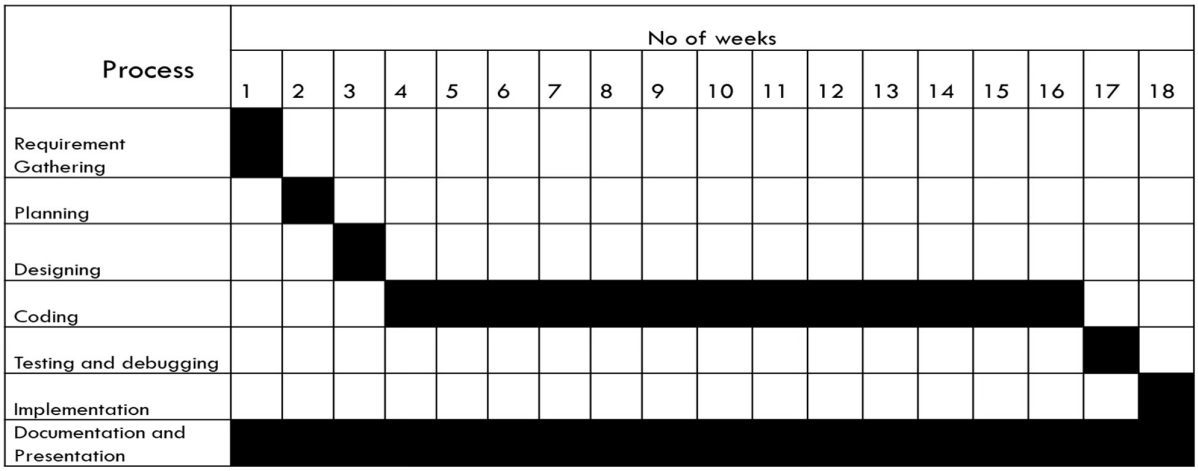

Figure 1: Gantt Chart

4.4. System Design

4.4.3.1. Applicable Tools and Technologies

4.4.1.1. Frontend Technologies

- **HTML**: Used for creating the structure and content of web pages.
- **CSS3**: Used for styling web pages, including layouts, colors, fonts, and responsive design.
- **Tailwind CSS**: A utility-first CSS framework to design responsive and modern interfaces efficiently.
- **React JS**: A JavaScript library for building dynamic, component-based user interfaces and handling front-end logic.

4.4.1.2. Backend Technologies

- **Node.js**: A JavaScript runtime environment used to build scalable server-side applications, it is also required to install frontend/backend libraries and frameworks.
- **Express.js**: A web application framework for Node.js that simplifies routing, middleware integration, and server-side logics.

4.4.1.3. Database Management

- **MongoDB**: A NoSQL database used to store structured and unstructured data, such as user profiles, job postings, applications, and transaction records.
- **Mongoose** ODM: An Object Data Modeling (ODM) library, specially designed for MongoDB to maps **JavaScript objects to MongoDB documents** instead of mapping tables and rows (as in relational databases).

4.4.1.4. Authentication and Security

- **JWT (JSON Web Token)**: Used for secure user authentication and session management. Tokens are generated upon login to verify and authorize users for accessing protected routes.

4.4.1.5. Payment Gateway Integration

- **eSewa**: A widely used online payment gateway in Nepal. It handles subscription payments for premium users and activates premium features upon successful transactions.

4.4.1.6. API Testing and Debugging

- **Postman API**: A tool to test backend APIs, verify endpoints, check request-response functionality, and debug the system during development.

4.4.1.7. Development Tools

- **Visual Studio Code (VS Code)**: A lightweight and versatile code editor used for writing, debugging, and managing both frontend and backend code.
- **Node.js Runtime Environment**: Provides the server-side environment needed to run JavaScript code and execute backend processes. It helps to manage the libraries and framework using Node Package Manager.

4.4.3.2. Applicable Algorithms

The KARYASETU system uses two main algorithms to enhance job matching: **Cosine Similarity** for evaluating how well a candidate matches a job, and the **Haversine formula** for location-based filtering of jobs. Both algorithms are computationally efficient and well-suited for real time web applications.

4.4.2.1. Cosine Similarity Algorithm

Cosine similarity is used to measure the similarity between two sets of data represented as vectors. In the context of KARYASETU, the two vectors are:

1. **Job profile vector** – contains features such as required skills, education level, and experience.
2. **Candidate profile vector** – contains the corresponding features from the job seeker's profile.

The similarity score ranges from **0 to 1**, where **1 means an exact match** and **0 means no similarity**. The higher the score, the more suitable the candidate is for the job.

Mathematical Formula:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\| \vec{A} \| \| \vec{B} \|}$$

Where:

- $\vec{A} \cdot \vec{B}$ is the dot product of the job vector and candidate vector
- $\| \vec{A} \|$ and $\| \vec{B} \|$ are the magnitudes (lengths) of the vector

How it works in KARYASETU:

1. Each skill, experience, or education is represented as a numeric value or binary flag in a vector.
2. When a job is posted, the system converts the job requirements into a vector.
3. Each candidate profile is also converted into a vector.
4. The cosine similarity formula is applied to calculate a **score for each candidate-job pair**.
5. Candidates are **ranked in descending order** of similarity, helping employers quickly identify the most relevant applicants and also this helps to list down recommended jobs based on similarity score in job seeker dashboard.

Example:

Job requires skills in: [Java, SQL, HTML, CSS] → vector [1, 1, 1, 0]

Candidate profile has skills [Java, SQL, Python] → vector [1, 1, 0, 0]

$$\text{Similarity} = \frac{(1*1)+(1*1)+(1*0)+(0*0)}{\sqrt{1+1+1+0}*\sqrt{1+1+0+0}} = \frac{2}{\sqrt{3}*\sqrt{2}} \approx 0.816$$

A similarity of **0.816** indicates a strong match, so the candidate would be ranked highly for this job.

4.4.2.2. Haversine Formula for Location-Based Filtering

The Haversine formula calculates the **great-circle distance** between two points on the Earth using latitude and longitude coordinates. This helps the system recommend jobs that are **geographically close** to the job seeker, improving the practicality of recommendations.

Mathematical Formula:

$$d = 2r \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

Where:

- d = distance between two points
- r = radius of the Earth (~6371 km)
- ϕ_1, ϕ_2 = latitudes of the two points in radians
- λ_1, λ_2 = longitudes of the two points in radians
- $\Delta\phi = \phi_2 - \phi_1$
- $\Delta\lambda = \lambda_2 - \lambda_1$

How it works in KARYASETU:

1. Each job has a location.
2. Each job seeker has a preferred location.
3. The Haversine formula calculates the distance between the job location and the seeker's location.
4. Jobs within a certain radius (e.g., 50 km) are prioritized in recommendations.

Example:

Job location: Kathmandu (Lat: 27.7172, Lon: 85.3240)

Candidate location: Lalitpur (Lat: 27.6720, Lon: 85.3160)

Applying the Haversine formula gives a distance of approximately **5 km**, making this job highly relevant for location-based recommendation.

4.4.3.3. Proposed Work

4.4.3.1. System Architecture

Software Architecture refers to the high-level structure and organization of a system. It defines how system components are designed, how they interact, and how data flows between them, serving as a blueprint for development and maintenance.

In the proposed system **KARYASETU – An Intelligent Job Portal**, the architecture follows a client–server-based web application model. The frontend is developed using ReactJS along with HTML, CSS, JavaScript, and Tailwind CSS to handle the user interface and interactions. The backend uses NodeJS with ExpressJS to manage server-side logic and client requests. Mongoose acts as an Object Data Modeling (ODM) tool to connect the backend with the MongoDB database for data storage and management. This architecture ensures clear separation of concerns, scalability, and easy maintenance.

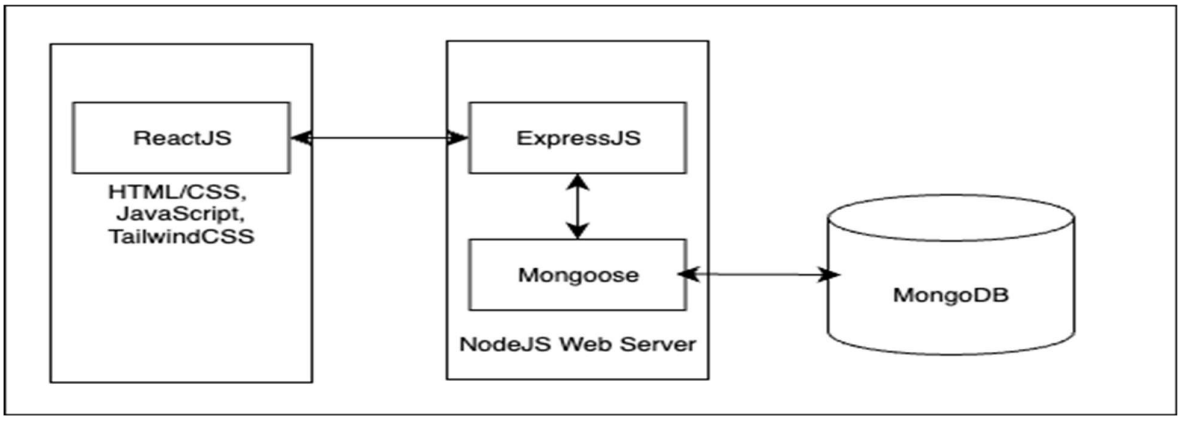

Figure 2: System Architecture of Our Proposed System

4.4.3.2. Flowchart Diagram

A flowchart **is a** diagram that represents the steps of a process or algorithm in sequence**,** using symbols and arrows to show the flow of actions from start to end.

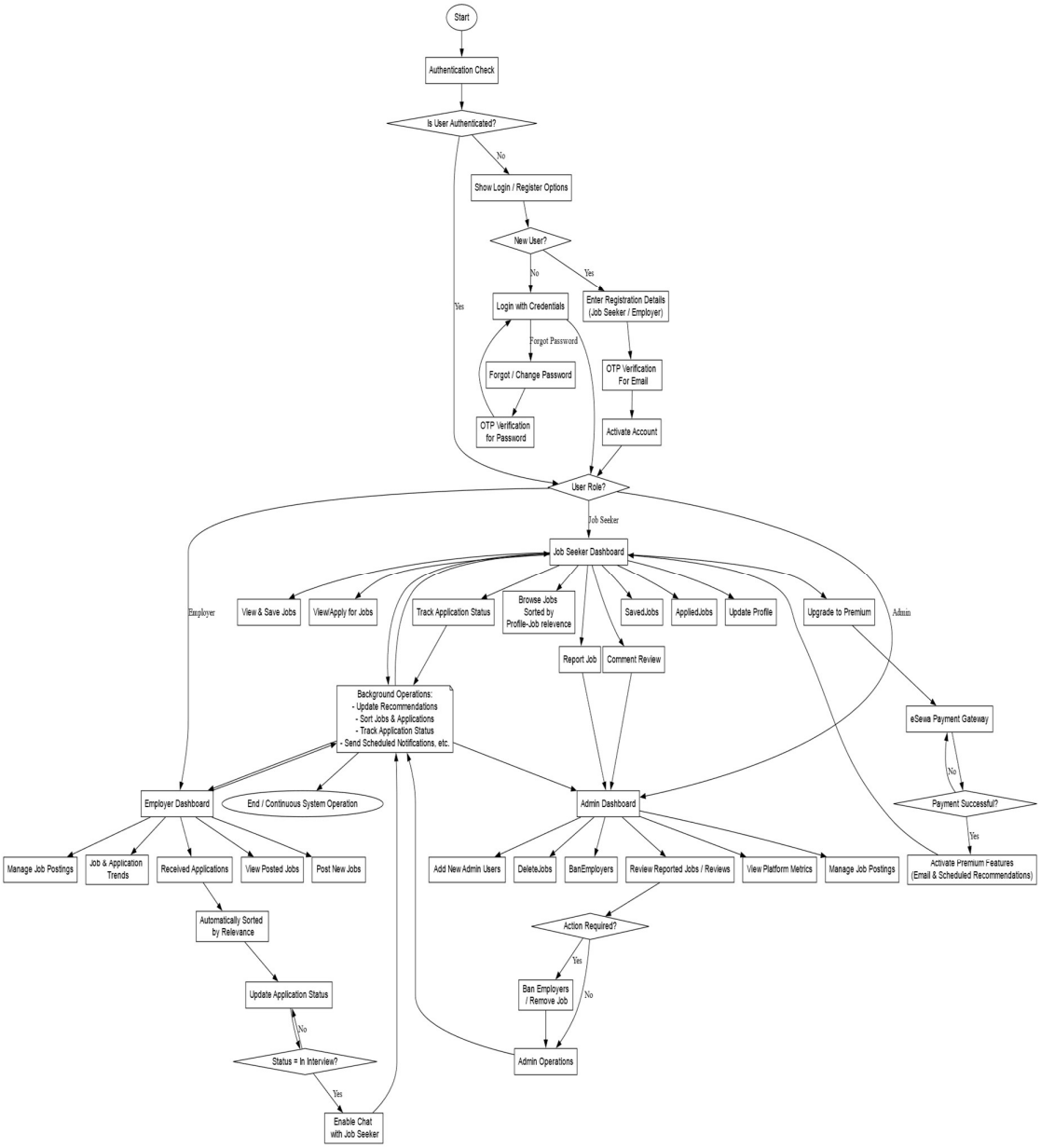

Figure 3: Flowchart Diagram of Proposed System

4.4.3.3. Use Case Diagram

A Use Case Diagram is a UML (Unified Modeling Language) diagram that visually represents how a user (actor) interacts with a system to achieve a specific goal**,** showing the system's functionality from the user's point of view.

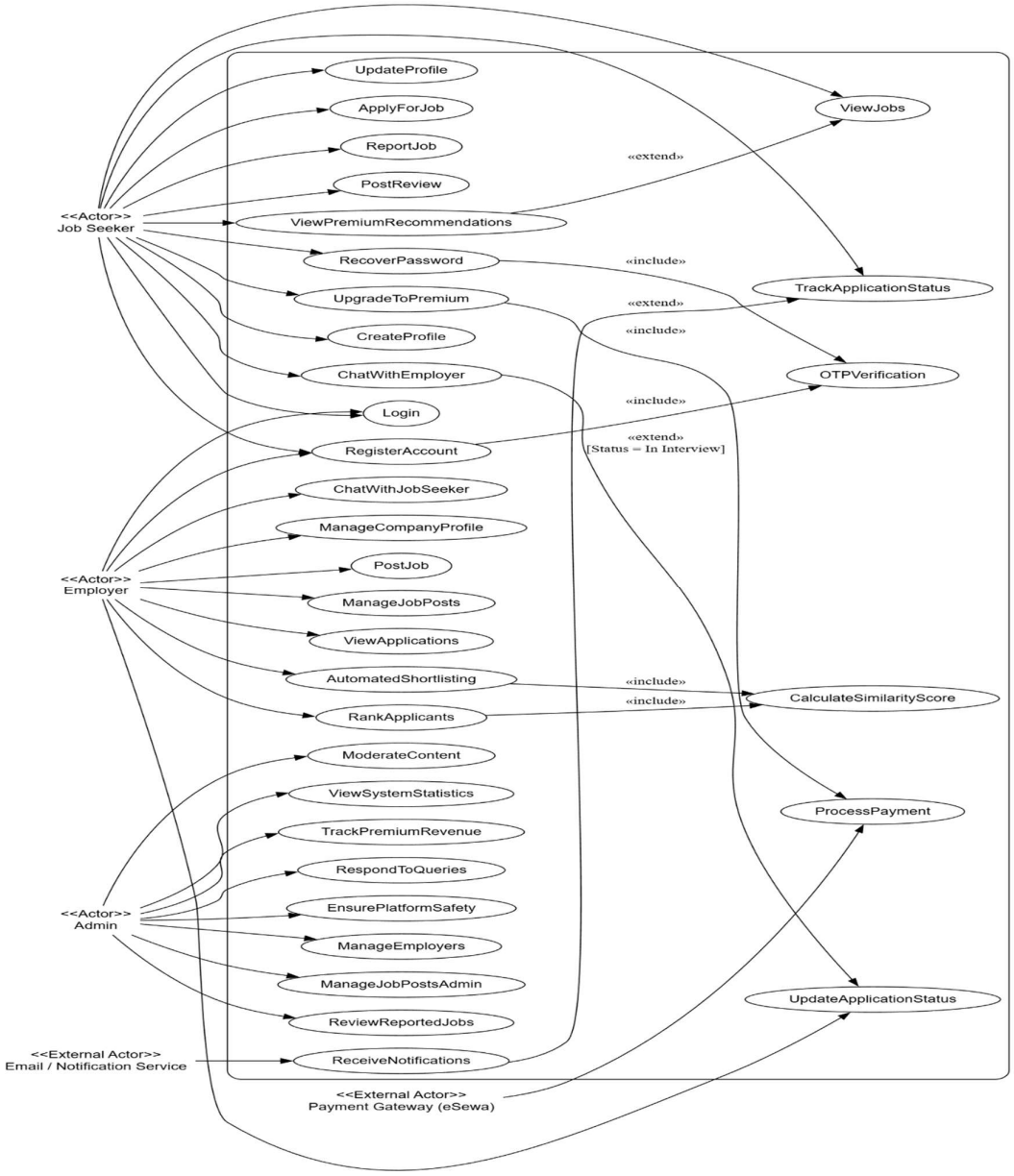

Figure 4: Use Case Diagram of Proposed System

4.4.3.4. MongoDB Data Model

A MongoDB data model diagram illustrates the structure of MongoDB's document-based data model by depicting collections, their documents, fields, and the relationships between documents using embedding or referencing, instead of tables and joins.

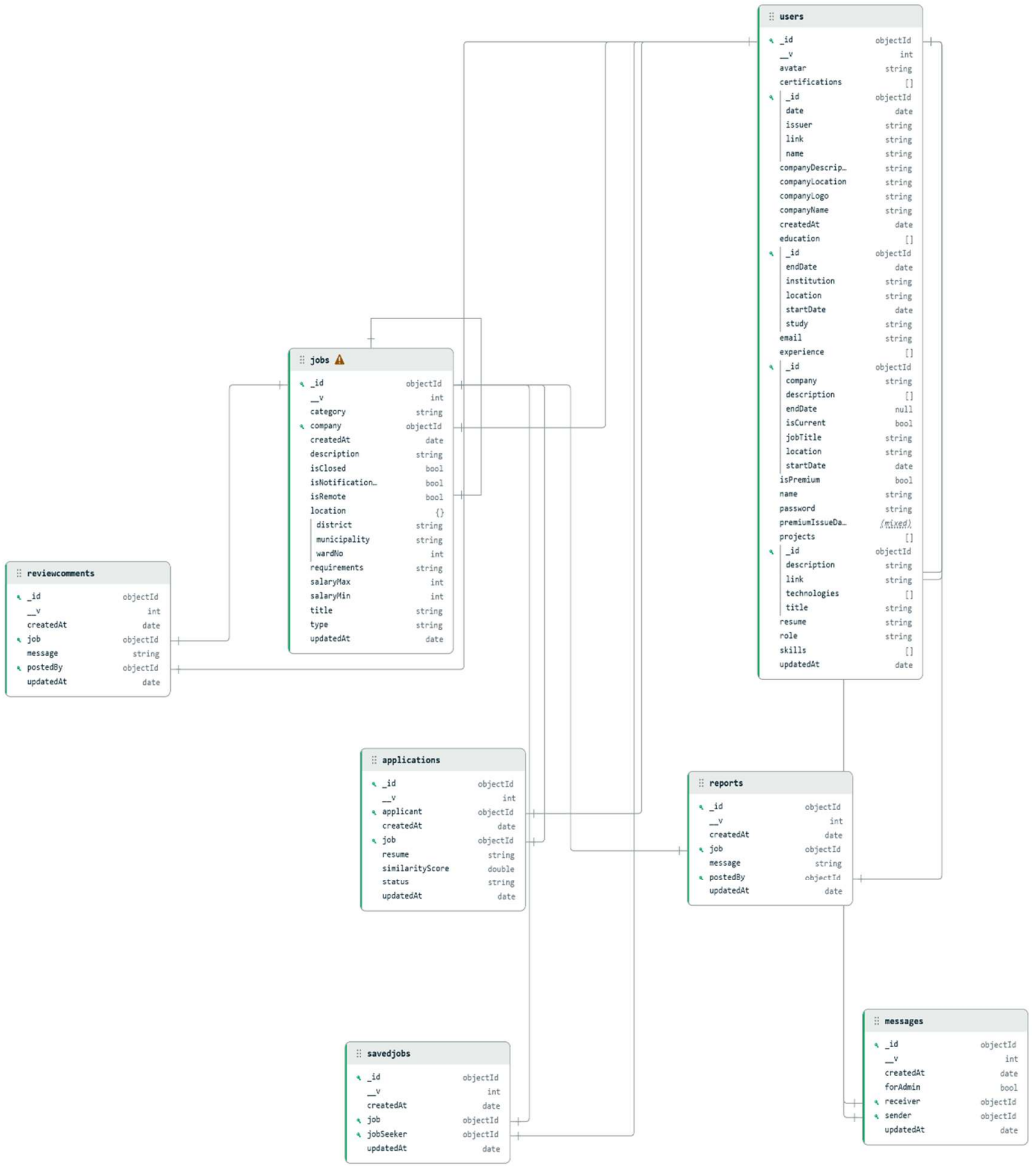

Figure 5: MongoDB data model of Proposed System

5. Expected Outcomes

After the completion of this project, we should be able to develop features mentioned below:

- **Efficient Job Matching:** Personalized job recommendations for seekers and ranked applicant lists for employers reduce manual resume review and speed up hiring process.
- **Resume Screening:** Cosine similarity is used to match candidate's profiles with job requirements, improving efficiency and reducing bias.
- **Location-Based Recommendations:** Jobs are prioritized using the Haversine formula, helping candidates find nearby and suitable opportunities.
- **Secure Authentication:** Email OTP verification, password change, and OTP-based recovery ensure strong account security.
- **Application Tracking:** Candidates can track application status (Applied, In Review, Interview, Hired). Premium users receive email updates.
- **Automated Recommendations:** Premium users receive up to 12 personalized job recommendation emails per month.
- **Structured Resume Data:** Skills, education, and experience are collected in a structured format for accurate matching.
- **Admin Panel:** Admins monitor system performance, manage content, handle reports, and track premium revenue.
- **Employer–Candidate Communication:** Secure in-app chat is available during the interview stage.
- **Job Safety & Moderation:** Users can report suspicious jobs, which admins review to maintain platform trust.
- **Payment & Subscription:** Premium features are enabled through **eSewa** payment integration.
- **Overall Impact:** Intelligent automation, secure processes, and location-aware matching modernize recruitment in Nepal.

6. References

[1] A. W. a. C. S. Y. G. Salton, "A vector space model for automatic indexing," *Communications of the ACM,* vol. 18, no. 11, p. 613–620, Nov. 1975.

[2] E. A. T. a. G. T. Faliagka, "An integrated e-recruitment system for automated personality mining and optimal candidate selection,," *Internet Research,* vol. 22, no. 5, p. 551–568, 2012.

[3] J. e. a. Malinowski, "Matching People and Jobs: A Bilateral Recommendation Approach," in *Proceedings of the 39th Annual Hawaii International Conference on System Sciences (HICSS'06)*, 2006.

[4] S. T. A.-O. a. M. Ykhlef, "A survey of job recommender systems," *International Journal of Physical Sciences,* vol. 7, no. 29, p. 5127–5142, 2012.

[5] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope,* vol. 68, no. 2, p. 159, 1984.